Simulation and Planning of Soft Growing Robots with Movable Objects

Anonymous Author(s)

***Abstract*— Soft growing robots, also known as vine robots, extend by everting from the tip, which lets them move through confined, cluttered spaces through their compliance. Existing models of vine robots have investigated navigation, where the environment is immovable, but not manipulation, where objects in the scene could be moved. This work first presents a vine simulator, built on an impulse-based contact formulation, in which rigid objects move according to their mass, inertia, and Coulomb friction in reponse to the vine's actuation. Second, building on this simulator, we propose an extension to prior GPU-parallel kinodynamic planning-as-design approaches to consider the pose of movable objects as well as multiple deployments, enabling multi-step manipulation planning. We demonstrate both the accuracy of our simulator against real trials and the effectiveness of our solver across simulated block puzzles. We release our implementation as open source.**

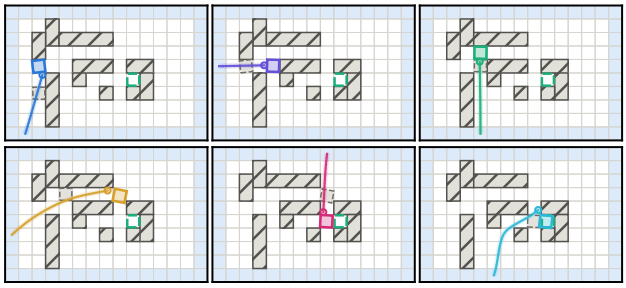


Fig. 1: A multi-step block pushing puzzle (5c) solved with our planning and simulation framework. Multiple vine deployments are used to push the block to the goal. The planner automatically exploits contact with the environment and block to find a solution.

I. Introduction

Soft growing robots, also known as vine robots, extend by pressure-driven eversion at the tip [1], and due to their compliant nature are therefore suited to confined narrow passages as in medical and inspection applications [2, 3]. While difficult to model, contacts between the vine and the environment have been studied in a number of contexts, such as exploiting them to steer the vine [4, 5] or reveal the shape of the environment [6]. However, vine models and planners have so far treated everything the vine contacts as immovable [7–9]. Vines can exert large forces [1], and they have been shown to grasp, wrap, or carry objects [10, 11] and to push compliant objects out of their way [12]. To our knowledge, no prior work predicts how a free object moves when a vine pushes it, or plans such pushes.

Adding movable objects allows us to explore many additional problems, of which we focus on two (Fig. 2). The first is payload pushing, moving an object to a goal, which is hard to plan even for rigid robots because where the object goes depends on its mass and friction, which may be poorly modeled [13, 14]. A vine's tip also everts, so new material rolls out over the contact instead of holding the object, and the object can slide off the tip as the vine bends. A single growth rarely places the object at the goal, so most tasks need several pushes, with the vine retracted between them [15]. The second is navigation among movable obstacles, or NAMO [16]: a vine that can push an obstacle aside, rather than only deflect off it, which may reveal paths to the goal that a planner with a fixed environment could never discover.

To this end, we present a simulator for vine robots among movable objects and a parallel kinodynamic planner that uses this simulator. The simulator is built on the impulse-based contact formulation of [7, 8], which includes mass and inertia, and the actuation model of [9] for series pneumatic artificial muscles (sPAMs) [17, 18]. To these we add movable rigid boxes with mass, inertia, and Coulomb friction, and growth that stalls at a bounded force, so that blocked vines push on movable objects. Our planner builds upon prior work that views sPAM-actuated vine design as a kinodynamic planning problem [9]; the planner is a kinodynamic RRT [19] that searches over these layouts by propagating batches of edges through the simulator in parallel on a GPU, sampling the tip and objects parameters to guide search. For tasks that need several pushes, our planner also plans resets of the vine between pushes, enabling deployment from another part of the workspace.

We evaluate the physical accuracy of our simulator on validation runs with a real, unactuated vine, achieving 16-18 mm vine error and 15-38 mm block error on simple contact, long-horizon, and puzzle experiments. We also demonstrate our planner on 15 problems in simulation, from single pushes to navigation through movable obstacles and block puzzles of up to six pushes (Fig. 1). Finally, we validate the feasibility of our plans in a real-world trial, executing the plan for puzzle 5b on the real vine. We release our implementation as open source.

II. Related Work

A. Vine Robot Modeling

Vine robots are thin, inextensible fabric tubes that extend by everting pressurized material at the tip [1, 20]. Under lateral contact, the body bends linearly until its compressed side wrinkles, after which the restoring moment saturates [21], and the tube kinks at a single point [22]. Models of this behavior typically trade fidelity for speed. Finite element models resolve the fabric and the internal pressure [23], but are too slow for planning, as thousands of candidate motions need to be evaluated to find feasible designs. Analytic models of the

vine's shape from beam theory or geometry [24–26] do not account for the forces it exerts, and lumped-parameter models capture growth without bending or contact [27]. Rigid-link models treat the vine as a chain of rigid bodies that grows at the tip [7], so contact with friction can be handled by standard rigid-body methods. Recent rigid-link simulators add sPAM actuation with parallel rollouts [9] and physically grounded contact and friction [8], both among fixed obstacles. We build on these ideas and add movable objects with mass, inertia, and friction (Sec. IV).

B. Kinodynamic Planning with Contact

Kinodynamic planning finds controls that take a system from a start to a goal under its dynamics [28]. Joining two states exactly requires solving a two-point boundary value problem, which has no closed form for most systems. Sampling-based kinodynamic planners instead sample controls and propagate them forward through the dynamics to grow a tree [19, 29]. Propagation dominates their cost, so recent planners parallelize it on GPUs [30–32]. Forward propagation also suits underactuated systems such as vines, whose only inputs are growth and a few sPAM curvatures, and pushed objects, which have no actuators of their own [13, 14]. In contrast to normal geometric planning techniques which reject edges that collide with the environment, vine robots are meant to touch obstacles and push objects; thus, we must simulate the vine forward in time to see how it deforms and modifies the environment.

Rigid robots push objects both to move them to goals and to clear them out of the way [16, 33]. Physics-based planners for these tasks grow trees whose edges are simulated pushes [34–37]. Our planner follows the same approach with movable objects in the state, and adds vine resets for tasks that take several pushes.

C. Vine Robots and Movable Objects

Vines can act strongly on objects—they exert large forces by growing against them [1], carry tools on a tip mount [38], wrap around an object to grasp and pull it [10], manipulate with deployable fingers at the tip [39], and transport objects along the everting body [11]. These interactions are mostly prehensile; where a vine pushes an object without grasping it, as in pushing a compliant object aside in a pipe junction [12], the behavior has been characterized in experiments but not predicted. Existing vine planners use contact, but with fixed obstacles: they sequence interactions [5] or find growth paths that need the least pressure [40]. To our knowledge, no prior work simulates the motion of a free object pushed by a vine, or plans such pushes to move a payload to a goal or to clear a path for the tip.

III. Problem Formulation

We formulate both payload pushing and navigation among movable obstacles as kinodynamic planning problems with our simulator as the dynamics. Consider a system with state $x \in \mathcal{X}$ and control $u \in \mathcal{U}$, whose dynamics $x_{t+1} = f(x_t, u_t)$ advance the state by a time step Δt. Here, f is one step of our simulator (Sec. IV). Given a start state x_0, a set of valid states $\mathcal{X}_{\text{valid}} \subseteq \mathcal{X}$, and a goal region $\mathcal{X}_G \subseteq \mathcal{X}$, the kinodynamic planning problem is to find a sequence of controls $u_{0:H-1}$ whose rollout from x_0 stays in $\mathcal{X}_{\text{valid}}$ and ends in $\mathcal{X}_G$.

The vine grows in a plane among static walls and K movable rigid boxes that rest on a table (Fig. 2). The state $x = (\boldsymbol{b}, \boldsymbol{s}, \boldsymbol{o}_1, \ldots, \boldsymbol{o}_K)$ consists of the base pose, the vine state, and the box states. The base pose, or launch pose, $\boldsymbol{b} \in \mathrm{SE}(2)$ is the position on the boundary of the workspace from which the vine grows and the heading at which it is launched. The vine state $\boldsymbol{s}$ contains the pose and velocity of each grown segment and the rest curvature built into it. Each box state $\boldsymbol{o}_k$ contains the pose $(\boldsymbol{c}_k, \psi_k)$ and velocity of box k, where $\boldsymbol{c}_k$ is its center and ψ_k its heading. The base stays fixed while the vine grows at a fixed speed $\bar{g}$, and the control $u = \kappa \in [-\kappa_{\max}, \kappa_{\max}]$ is the curvature of the material that grows while the control is applied. A state is valid if no two bodies interpenetrate and the vine is no longer than its maximum length.

Following Gao et al. [9], we hold each control for T steps, so a plan is a sequence of approximately constant-curvature pieces. Grown material keeps the curvature it was grown with, so piece j is an sPAM segment of length $\bar{g}\,T\,\Delta t$ that starts where piece $j-1$ ends and is fabricated for curvature κ_j. A plan is therefore an actuator layout.

A vine without sPAMs grows straight ($\kappa = 0$) until something deflects it. Its only decision is its base pose $\boldsymbol{b}$, so a plan is a base pose for each push, and the walls and boxes do the steering. Our real experiments and the block puzzles use unactuated vines.

Let $\boldsymbol{x}_{\text{tip}}(x)$ be the tip position in state x and $\mathcal{G}$ a goal disc in the plane. The two problems differ only in their goal region:

1) *Payload pushing*: move a target box $k^\star$ into the goal, $\mathcal{X}_G = \{x \mid \boldsymbol{c}_{k^\star}(x) \in \mathcal{G}\}$.
2) *Navigation among movable obstacles*: move the tip into the goal, $\mathcal{X}_G = \{x \mid \boldsymbol{x}_{\text{tip}}(x) \in \mathcal{G}\}$, where some or all of the obstacles are movable boxes.

Growth is irreversible, so there are limited pushes that a single vine can apply to the objects. For tasks that need more, a plan is a sequence of pushes separated by resets. A reset retracts the vine to its starting state $\boldsymbol{s}_0$ and leaves the boxes where the previous push left them, mapping $x = (\boldsymbol{b}, \boldsymbol{s}, \boldsymbol{o}_{1:K})$ to $(\boldsymbol{b}', \boldsymbol{s}_0, \boldsymbol{o}_{1:K})$. In simulation, we set the vine state to $\boldsymbol{s}_0$ rather than simulate its retraction. For more complicated puzzles, we also allow the base pose to change in between pushes.

IV. Methodology

A. Simulation and Contact Formulation

Predicting a push requires knowing how far a block slides, whether it turns, and when it stops, which depends on the block's mass and on friction forces that scale with the normal force. We therefore build on the velocity-level formulation of [7, 8], in which every body has mass and inertia and every contact is resolved by an impulse and extend it to movable objects.

As in Chen et al. [8], the vine is a chain of rigid bodies in maximal coordinates, each a circle of the vine's radius with a

planar pose, and it grows by lengthening its distal link. Each box adds its pose (c_k, ψ_k), and walls are fixed oriented boxes. Let $\boldsymbol{v}$ stack the velocities of all vine bodies and boxes. Each step solves the quadratic program (QP)

$$\min_{\boldsymbol{v}} \ \tfrac{1}{2}\boldsymbol{v}^\top \boldsymbol{M}\boldsymbol{v} + \boldsymbol{p}^\top \boldsymbol{v} \quad \text{s.t.} \quad \boldsymbol{G}\boldsymbol{v} \le \boldsymbol{h}, \quad \boldsymbol{A}\boldsymbol{v} = \boldsymbol{b}, \quad (1)$$

and advances every pose by $\Delta t\, \boldsymbol{v}$. The mass matrix $\boldsymbol{M}$ is block diagonal in the masses and inertias of the vine bodies and of the boxes. The linear term $\boldsymbol{p} = \Delta t\, \boldsymbol{\tau} - \boldsymbol{M}\boldsymbol{v}_t$ carries the momentum of the previous step and the bending torques $\boldsymbol{\tau}$ from the vine's stiffness, damping, and sPAM actuation [9].

The equalities $\boldsymbol{A}\boldsymbol{v} = \boldsymbol{b}$ act on the vine alone. Two constraints per joint require the ends of consecutive bodies to coincide after the step, and a growth constraint requires the distal link to lengthen at the growth speed $\bar{g}$, the rate at which the real vine everts material at its tip. A blocked vine cannot meet this constraint, so we relax it with a slack s: the distal link must lengthen at $\bar{g} - s$, and a penalty $\frac{1}{2} c\, s^2$ is added to the objective, acting like a spring pulling the growth speed back toward $\bar{g}$. A free vine grows at $\bar{g}$; a vine held back pushes on whatever resists it with a force that rises with the shortfall, capped at a growth force F_{grow} set by the penalty when the tip is stopped.

The inequalities $\boldsymbol{G}\boldsymbol{v} \le \boldsymbol{h}$ hold one non-penetration constraint per pair of bodies that may touch: a vine body with a wall or a box, and a box with a wall or another box. Each linearizes the pair's gap ϕ_j so that it remains nonnegative after the step, $\phi_j + \Delta t\, \boldsymbol{J}_j \boldsymbol{v} \ge 0$, where $\boldsymbol{J}_j$, the gradient of ϕ_j with respect to the poses of the two bodies, is computed by automatic differentiation. This is all that movable objects require: a box is three more columns of $\boldsymbol{M}$ and $\boldsymbol{G}$, and a vine–box constraint has entries for both the vine body and the box, including the box's rotation. Its multiplier $\lambda_j \ge 0$ is the contact impulse, applied equally and oppositely to the two bodies, so the box receives exactly the impulse, and the torque about its center, that keeps the vine from penetrating it. Friction (Sec. IV-B) splits each object contact constraint into two such inequalities and adds block–table friction to the objective; it adds no equalities.

Chen et al. [8] solve Eq. (1) with cvxpylayers [41], one program at a time on the CPU; we instead solve the programs of a whole batch of rollouts at once on the GPU with a primal–dual interior-point method, each iteration of which is a single batched factorization. This solver is not differentiable, so rather than fitting the simulator's parameters by backpropagation as in Chen et al. [8], we fit them to tracked videos with the cross-entropy method [42], simulating each generation's population of parameter sets as one batch.

B. Friction Contact Formulation

Vine models usually omit friction, since a vine sliding along a fixed wall is steered mainly by the wall's normal force. A pushed block, however, slides, turns, or stays put depending on how hard the vine, the walls, the other blocks, and the table grip it, so we add Coulomb friction to vine–block, block–wall, block–block, and block–table contact; vine–wall contact stays frictionless.

Algorithm 1: Batched kinodynamic RRT

Input: start x_0, goal region $\mathcal{X}_G$, batch size B
Output: controls $u_{0:H-1}$

$\mathcal{T} \leftarrow \{x_0\}$
while *not solved* **do**
  for $i \leftarrow 1$ **to** B **do**
    $x_{\text{rand}} \leftarrow$ `Sample()`
    $x_i \leftarrow$ `Nearest`$(\mathcal{T}, x_{\text{rand}})$
  $x'_i \leftarrow f(x_i, u_i),\ u_i \sim \mathcal{U},\ i = 1, \ldots, B$ `// one batched call`
  foreach $x'_i \in \mathcal{X}_{valid}$ **do**
    add x'_i to $\mathcal{T}$ as a child of x_i with control u_i
    if *retractable* **then** `TryReset`$(\mathcal{T}, x'_i)$ `// Sec. IV-D`
return *the controls on the path from* x_0 *to* $x_i \in \mathcal{X}_G$

a) Contact friction: Coulomb friction with coefficient μ requires each contact impulse to lie in the friction cone, $|f_t| \le \mu f_n$, and to oppose slip. Classic time-stepping methods [43, 44] pose this as a linear complementarity problem, which is not the optimality condition of a convex program and so cannot be solved as a QP. To keep each step a single QP, we use the convex relaxation of [45]: in the plane the cone has two edges, $\boldsymbol{n}_j \pm \mu \boldsymbol{t}_j$, and each becomes a non-penetration constraint of its own,

$$\phi_j + \Delta t\, (\boldsymbol{J}_{n,j} \pm \mu \boldsymbol{J}_{t,j})\boldsymbol{v} \ge 0, \quad (2)$$

where $\boldsymbol{J}_{n,j}$ (the $\boldsymbol{J}_j$ of Sec. IV-A) and $\boldsymbol{J}_{t,j}$ map $\boldsymbol{v}$ to the relative velocity of the two contact points along the normal $\boldsymbol{n}_j$ and the tangent $\boldsymbol{t}_j$. Each constraint pushes along one edge of the cone, so their combined impulse always lies inside it. When the push is too weak to overcome static friction, both constraints are active and the contact sticks; otherwise only the one facing the slip is active, and the contact slides with friction μ times the normal impulse, opposing the slip. A side effect of the relaxation is that sliding bodies drift slightly apart, at a rate proportional to μ and their sliding speed.

The three contact types differ only in their tangential velocity and in their coefficient, each fitted to video data. For vine–block contact it is the velocity of the block's surface point nearest the vine body, which includes the block's rotation, so friction resists the block turning under the vine as well as sliding along it. For block–wall and block–block contact it is the velocity of each block's support point toward the contact, so friction both slows a block and turns it.

b) Block–table friction: The table's normal force on block k is its weight $m_k g$ rather than an unknown contact impulse, so instead of a constraint we add standard Coulomb friction with coefficient μ_{bt} against the block's sliding and turning as a cost in the objective, approximated by polygons so that the step remains a QP.

C. Vine and Block Planning

We plan for solution controls $u_{0:H-1}$ with a batched kinodynamic RRT (Alg. 1). As in standard kinodynamic

Algorithm 2: Hierarchical block puzzle search

Input: start x_0, goal cell of the target block
Output: base pose $\boldsymbol{b}$ and growth length of each push

```
1  S ← [x_0]            // states reached so far
2  while not solved and S ≠ ∅ do
3      x ← top of S
4      π ← CellPlan(x)      // Dijkstra over block cells
5      if π = ∅ then
6          pop S; ban the move that led to x
7          continue
8      (ok, x') ← PushSearch(x, π)    // batched sim
9      if ok then push x' on S
10     else ban the first move of π from x
11 if S = ∅ then return failure
12 return S
```

RRT [19], each expansion samples a state x_{rand} (line 4), finds the node x_i of the tree $\mathcal{T}$ nearest to it (line 5), then samples a control $u_i \in \mathcal{U}$ and simulates it from x_i (line 6). Valid results are added to the tree, with the control on the edge from parent to child (lines 7–8). The batched planner does each step for B expansions at once, and the B rollouts are a single batched call to the simulator on the GPU. The search ends once a node reaches the goal region, and the plan is the controls along its path (line 10).

Contact does not invalidate a state, since the simulator resolves it. Beyond $\mathcal{X}_{\text{valid}}$, a rollout is discarded if the vine or a box reaches the border of the workspace or the solver fails, and a node whose expansions fail to grow the vine three times is no longer selected for expansion.

Like classical rigid-body non-prehensile manipulation, the box states $\boldsymbol{o}_1, \ldots, \boldsymbol{o}_K$ are part of the state x, which greatly increases its dimension for problems with many boxes, such as 4d (Fig. 2). The dimension of $\mathcal{X}$ also grows with the vine's length, and most of it, the vine state $\boldsymbol{s}$, is determined by the controls already applied. We therefore sample and measure distance only in the projection of x onto the tip, $\boldsymbol{x}_{\text{tip}}(x)$ (problem 4a), or onto the tip and the box centers, $(\boldsymbol{x}_{\text{tip}}(x), \boldsymbol{c}_1(x), \ldots, \boldsymbol{c}_K(x))$ (lines 4–5). For retractable vines, each new node may also lead to a reset (line 9, Sec. IV-D).

D. Multi-Push Planning

A single push often cannot move a block far without a wall or another obstacle to guide it, and growth is irreversible, so these tasks need several pushes separated by resets (Sec. III). Since we lack a model of retraction, a reset sets the vine state back to $\boldsymbol{s}_0$ in simulation, and the real vine is retracted and relaunched by hand. We plan multiple pushes in two ways: resets inside the RRT for retractable vines, and a hierarchical search for the block puzzles.

a) Resets in the tree: Restarting the tree for each push is greedy: it commits to the previous push before a global solution is found and tends to get stuck in local minima. We instead keep a single tree in which a reset is an edge like any growth step. Allowing a reset after every expansion would make the tree needlessly dense, since a fresh vine needs several growth steps before it reaches a box again, and a fixed reset probability spends resets where the boxes have not moved. `TryReset` (Alg. 1, line 9) therefore resets a new node $x' = (\boldsymbol{b}, \boldsymbol{s}, \boldsymbol{o}_{1:K})$ only once its boxes have moved at least δ from every box configuration already reset at, a box's motion being its center's shift plus its turn times its half diagonal. We set $\delta = 50$ mm, about half a block's width, so that a reset follows a push that has moved a block substantially, not one that merely nudged it. The reset adds the child $(\boldsymbol{b}, \boldsymbol{s}_0, \boldsymbol{o}_{1:K})$, a fresh vine with the boxes where x' left them, and records $\boldsymbol{o}_{1:K}$, so that later nodes must move the boxes further before resetting again.

b) Hierarchical Search: In the block puzzles the blocks sit on a grid of cells between walls, the vine is unactuated, and the base pose $\boldsymbol{b}$ changes between pushes, so a plan is a base pose and a growth length for each push. Alg. 2 is a hybrid search- and sampling-based planner. The high level discretizes the state and action spaces: a state is the cell each block occupies, and an action is an object-centric move that pushes one block along a grid axis into free cells, accounting for a clear path for the vine to push from. `CellPlan` (line 4) searches this graph, from the cells the blocks of x occupy and skipping banned moves, for a sequence of moves to the goal. The low level, `PushSearch` (line 8), realizes the plan's first move in the continuous space with the simulator. Since an unactuated vine's only free choice is its launch pose, it samples launch poses along the boundary, simulates them in one batch, and accepts a push that leaves the block at rest in the planned cell; an actuated vine would also sample curvatures, as in Alg. 1. An accepted push is applied and the high level replans from the new configuration (line 9); a failed one bans that move (line 10), and when no plan remains the search backtracks and bans the move that led there (line 6). Because the search can backtrack, no push is committed until a full plan reaches the goal. If it backtracks past the start, every option is exhausted and the search fails (line 2). The resulting solution is pulled from the stack (line 12).

V. Experiments

All experiments are conducted on a virtual machine equipped with an NVIDIA RTX 4090 GPU with 24 GB of VRAM, 64 GB of system memory, and 12 CPU cores allocated from a 24-core AMD Ryzen Threadripper PRO 5965WX processor operating at 3.8 GHz.

Planning parameters such as batch size are tuned per problem for the best solve times, as higher object counts affect the GPU's batch throughput limit. Simulator parameters were not fine-tuned to each problem, but different versions trained on relevant demo videos are used for subsets of the simulation, validation, and puzzle experiments and are specified in each section.

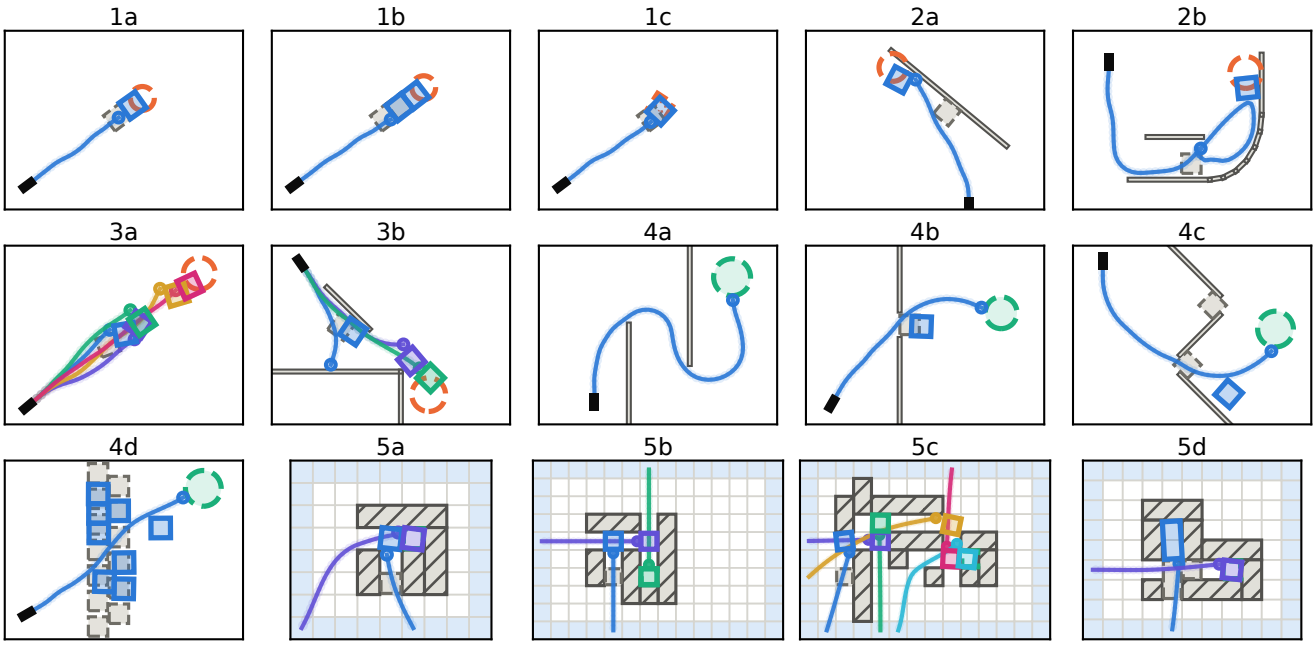


Fig. 2: All 15 planning problems depicted with solutions. Black bar: vine base; grey dashed box: initial block positions; dashed circle: goal (orange: block-to-goal, green: vine-to-goal). Each push's vine and the blocks it moved are drawn in the color of its order: first, second, third, fourth, fifth, and sixth.

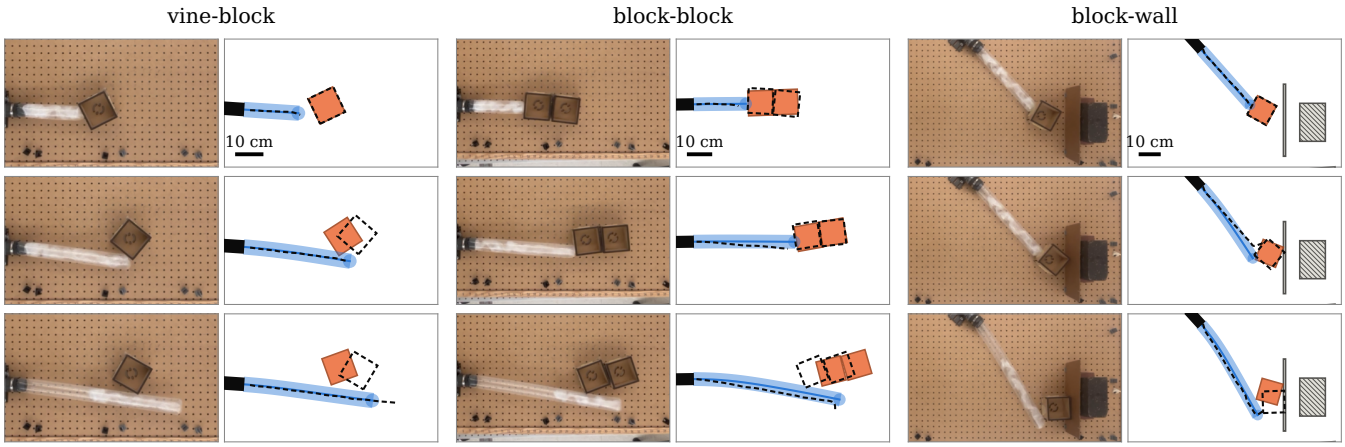


Fig. 3: Comparison of simulated and real runs of three simple contact scenarios. The validation videos were not included in the training data when fitting the simulator. Real vine and block positions are overlaid on the simulation visuals

A. *Planning Experiments*

Simulator experiments can be largely divided into four categories: basic block-to-goal, block-to-goal with retraction for multiple pushes, vine-to-goal with movable obstacles, and hierarchical block puzzles. Figure 2 illustrates all environments with solution trajectories, and Table I reports success rates, compute times, and the number of pushes.

The simulator used in these experiments was fit to sPAM-actuated vine rollouts from the simulator in [9], 15 real brick-push videos, and the 29 simple-contact training videos of Sec. V-B; for the puzzle problems, the block's torsional friction was additionally tuned by hand on 10 real block puzzle videos.

1) Basic Block-to-goal: This problem suite demonstrates basic capabilities of our simulator and planner. 1a tests basic vine-block contact, 1b tests block-block contact, and 1c is a more precise goal condition where the orientation of the block matters. Furthermore, 2a and 2b test vine-wall and block-wall contacts in increasingly complex interactions. Note that without walls to guide the vine and block, we cannot set the goal too far away or else the planner fails to find a single push that solves the problem. The suspected cause is that as the vine tip everts, it is rolling along the surface rather than sliding, which negates much of the friction during pushes and tending to diverge to the sides.

2) Retractable Block-to-goal: This section tests our retractable vine planner and its ability to solve long-horizon tasks that require multiple pushes. As mentioned in Sec. IV-D, we do not simulate the retraction of a real life vine; rather, we revert the vine in simulation to its initial pose. 3a tests

Table I: Planning results over 20 runs per problem for the single push, retractable, vine-goal, and puzzle environments depicted in Fig. 2. Times, simulated edges, and pushes are over successful runs.

	Problem	Success	Time [s]	Edges [k]	Pushes
single	1a	20/20	17 ± 1	2.0 ± 0.1	
	1b	20/20	21 ± 10	2.2 ± 1.0	
	1c	20/20	20 ± 9	2.3 ± 1.0	
	2a	20/20	24 ± 1	2.5 ± 0.1	
	2b	20/20	84 ± 38	8.0 ± 3.3	
retr.	3a	14/20	759 ± 317	75.9 ± 30.9	6.8 ± 1.5
	3b	16/20	382 ± 392	34.2 ± 34.9	2.0 ± 0.6
vine-goal	4a	20/20	50 ± 5	6.0 ± 0.4	
	4b	20/20	31 ± 2	3.4 ± 0.1	
	4c	20/20	46 ± 14	4.7 ± 1.3	
	4d	20/20	91 ± 1	3.6 ± 0.0	
puzzle	5a	20/20	47 ± 3	0.45	2
	5b	20/20	109 ± 2	0.90	3
	5c	20/20	149 ± 16	1.10	6
	5d	20/20	34 ± 1	0.26	2

the basic multi-push problem in an open environment, and 3b provides wall interactions while acting as a divider to potentially divert retractions for a better push angle.

3) Vine-to-goal: These problems emulate scenarios where the vine must push an obstacle out of the way to reach a goal. 4a has no blocks and simply tests the vine's ability to reach a goal, while 4b is a basic wall with a movable obstacle blocking the doorway. 4c adds a layer of complexity with 2 blocks obstructing the gaps, but only the bottom block can be pushed in the direction the vine is heading. Finally, 4d tests the planner and solver's ability to handle a high-dimensional system with 13 obstacles.

4) Hierarchical Block Puzzles: As an extension to our multiple-push problems, we developed a more constrained discrete-space block push problem set, reminiscent of a block puzzle game. These problems exploit a hierarchical planner where the high level planner accounts for discrete object-centric motions to come up with a global plan before the low level planner decides specific, continuous actions for the vine to push the block to reach subgoals generated by the global plan. 5a-c incrementally ramps up difficulty, with 5c requiring diagonal vine launches. 5d explores an ordered problem with two blocks, with one blocking access to the other.

Unlike the retractable multi-push scenario, vines are not constrained to the same launch position and heading angle. Furthermore, to evaluate these problems in real life, no actuation is provided; instead, the walls guide the vine and block in a way that is both beneficial but also restricts the possible actions that result in useful pushes. The high level planner must account for both a path for the block to travel and an opening on the other side for the vine to push from.

B. Real World Validation

To measure the simulator accuracy, we compare its output to a live setup. Live experiments were conducted using a vine robot made from low-density polyethylene (LDPE) tubing, 76.2 mm flat with a thickness of 76.2 μm, resulting in a radius of ~24.3 mm. The vine operated at approximated 13 kPa psi and had a maximum length of one meter, which was never reached during the trials. Due to fabrication challenges, we did not use sPAM actuation for live trials.

Table II: Simulator error against real validation runs [mm]: mean distance between simulated and tracked vine and block centers. Each simulator adds training data to the last: the simulator output of vine with actuation in [9] only; + 15 real videos of general contact behavior; + real videos of the validation's scene type.

	Vine–block		Block–block		Block–wall		Long		Puzzle 5b	
Training data	vine	block	vine	block	vine	block	vine	block	vine	block
Sim only	50.0	85.6	45.3	132.0	16.8	29.5	72.4	17.9	23.1	26.2
+ general videos	**22.1**	66.8	**11.9**	69.2	13.7	21.2	25.2	**11.6**	21.2	26.0
+ scene videos	23.2	**55.7**	19.3	**41.3**	**11.8**	**18.3**	**16.2**	15.3	**16.1**	**24.8**

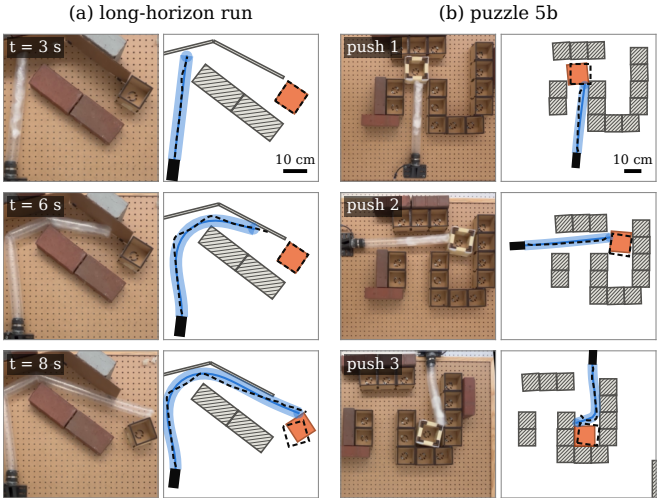


Fig. 4: Comparison of simulated and real runs of a long-horizon multi-contact scenario (left) and multi-push puzzle problem 5b (right). For puzzle 5b, the vine base was manually moved between pushes. The simulation environment obstacle positions were derived from the demo video rather than from Sec. V-A

We validated on three simple contact scenarios, a long-horizon push scenario with multiple wall contacts, and on puzzle problem 5b. Table II displays vine and block errors of validation experiments for simulators trained on various sets of data.

1) Simple Contact Validation: We validate our simulator on three simple contact scenarios in real life: vine-block, block-block, and vine-block-wall contacts. We trained the simulator on 29 videos of these scenarios (10 vine–block, 8 block–block, and 11 block–wall), then compared the results with one video of each category not included in the training set. Fig. 3 displays the real and simulated results side by side, with an overlay of the annotated real vine and block positions.

2) Puzzle Push Validation: We ran the straight-push solution of puzzle 5b (Fig. 2) on the real vine, moving its base by hand between the three pushes, and tracked each push as before (Fig. 4b). We simulated each push from its measured launch pose and starting block pose, using the same simulator

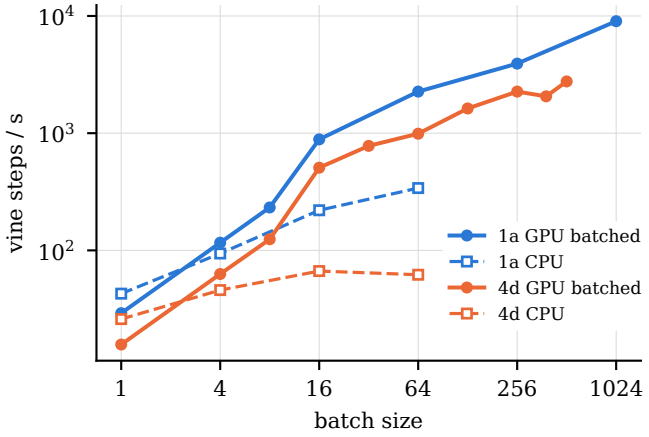


Fig. 5: Simulator throughput against batch size: vine steps per second with our batched GPU solver and with the per-element CPU solver (cvxpylayers). 1a has one block, 4d has 13 and stops at 512, beyond which its blocks exceed the GPU's memory.

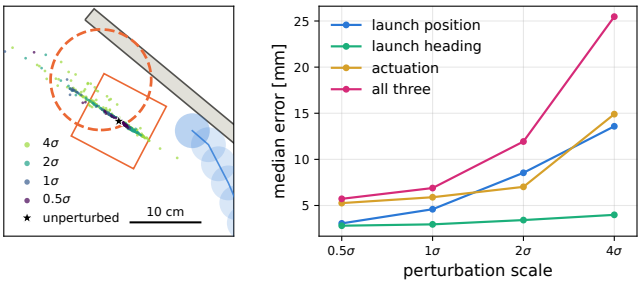


Fig. 6: Robustness of a 2a solution to perturbed execution. Left: final block position after perturbed rollout; black star: the unperturbed block position. Right: median block position difference against the perturbation scale, in multiples of σ.

that planned 5b. Over the frames the simulated vine reaches, the vine is within 5–8 mm and the block within 4–13 mm of the tracked ones, and the block ends each push within 26 mm of where the real one stopped.

3) Long Horizon Live Experiments: Without actuation, a run is fixed by its launch pose, so the long-horizon runs test the simulator alone over several contacts in sequence: the vine glances off two walls before it reaches and pushes the block. We train the scene-specific simulator on nine videos; on the validation run (Fig. 4a) it reduces the vine error from 72 mm with simulation data alone to 16 mm (Table II).

C. Ablations

1) Simulator Throughput: Figure 5 shows the simulator's batch efficiency. For a single rollout the CPU solver is faster, but the batched solver overtakes it from a batch of 4–8 and keeps scaling: on 1a it reaches 9,000 vine steps per second at a batch of 1024, 26 times the CPU's best, and on 4d 2,900 at 512, 43 times. More blocks mean more contact constraints per element, which lowers throughput and, for problem 4d, caps the batch through GPU memory; this is why batch sizes are tuned per problem.

2) Robustness to Execution Error: A short study was conducted to quantify the system's robustness to execution error. As posited in [9], vine robots display a funneling behavior during contact, often guided to the same side of the obstacle. We replay the 2a solution open loop with zero-mean Gaussian noise on the launch position, the launch heading, and the curvature of each piece, with standard deviations σ of 5 mm, 2°, and 0.5 m^{-1}. We scale these by 0.5, 1, 2, and 4, apply each perturbation alone and all three together, and run 256 samples of each, letting the block come to rest after the planned growth. Large perturbations cause little variation in the block's final position (Fig. 6): up to 2σ, the median resting block lies within 12 mm of the unperturbed one even with all three perturbations together.

VI. Conclusion

We presented a simulation and planning framework for vine robots with movable objects. The simulator extends a contact implicit rigid-body vine model to free-moving boxes and is GPU parallelizable. The planner, a batched kinodynamic RRT, builds on this simulator to plan for non-prehensile manipulation with vine robots on single push tasks, multiple pushes with retraction, and a hybrid hierarchical planner for multi-push puzzle problems, showing that vine robots are capable of achieving complex rearrangement tasks. Across real validation runs, from single contacts to a long-horizon run and a three-push puzzle, the simulator achieves accuracies of 12–23 mm for the vine and 15–56 mm for block positions, accurate enough for repeatable use and within the radius of the vine itself. Its main limitations are the tip contact, where the everting tip causes a partial rolling contact that is difficult to model with our friction formulation, and the fitted growth and bending models, whose errors in the vine's shape accumulate over long runs.

Future work includes the development of a contact model for the everting tip, which would address the main limitation identified above. We are also interested in planning with more complex object models, deformable objects, or granular material in the environment. Additionally, validation should be extended to experiments with sPAM-actuated vines, in contrast to the unactuated vines used in the real experiments this study. Such experiments would more closely match the system modeled by the simulator and would permit the vine to move in ways that depend less on its environment.

VII. Acknowledgment

ChatGPT/Codex and Claude code were used for language polishing and editorial assistance. All technical content, equations, experimental results, and final manuscript text were reviewed and verified by the authors.

References

[1] E. W. Hawkes, L. H. Blumenschein, J. D. Greer, and A. M. Okamura, "A soft robot that navigates its environment through growth", *Sci. Robot.*, vol. 2, no. 8, eaan3028, 2017.

[2] P. Berthet-Rayne, S. M. H. Sadati, G. Petrou, N. Patel, S. Giannarou, D. R. Leff, and C. Bergeles, "MAMMOBOT: A miniature steerable soft growing robot for early breast cancer detection", *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 5056–5063, 2021.

[3] Y. Qin, A. M. Okamura, and M. Salathe, "Design, control, and modeling for soft growing robot deployment for nuclear material inspection", in *INMM Annu. Meeting*, 2024.

[4] J. D. Greer, L. H. Blumenschein, R. Alterovitz, E. W. Hawkes, and A. M. Okamura, "Robust navigation of a soft growing robot by exploiting contact with the environment", *Int. J. Robot. Res.*, vol. 39, no. 14, pp. 1724–1738, 2020.

[5] M. Selvaggio, L. A. Ramirez, N. D. Naclerio, B. Siciliano, and E. W. Hawkes, "An obstacle-interaction planning method for navigation of actuated vine robots", in *IEEE Int. Conf. Robot. Autom.*, 2020, pp. 3227–3233.

[6] F. Fuentes and L. H. Blumenschein, "Mapping unknown environments through passive deformation of soft, growing robots", in *IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2023, pp. 2522–2527.

[7] R. Jitosho, N. Agharese, A. Okamura, and Z. Manchester, "A dynamics simulator for soft growing robots", in *IEEE Int. Conf. Robot. Autom.*, 2021, pp. 11 775–11 781.

[8] L. Chen, Y. Gao, S. Wang, F. Fuentes, L. H. Blumenschein, and Z. Kingston, "Physics-grounded differentiable simulation for soft growing robots", in *IEEE Int. Conf. Soft Robot.*, 2025, pp. 1–8.

[9] Y. Gao, L. Chen, P. Bhovad, S. Wang, Z. Kingston, and L. H. Blumenschein, "Parallel simulation of contact and actuation for soft growing robots", *Soft Robot.*, vol. 13, no. 3, pp. 405–420, 2026.

[10] K. Barhydt, O. G. Osele, S. Kodali, C. du Pasquier, C. M. Hartquist, H. H. Asada, and A. M. Okamura, "Loop closure grasping: Topological transformations enable strong, gentle, and versatile grasps", *Sci. Adv.*, vol. 11, eady9581, 2025.

[11] E. DeVries, J. Ferlazzo, M. Ugur, and L. H. Blumenschein, "Transport and delivery of objects with a soft everting robot", *IEEE Robot. Autom. Lett.*, vol. 11, no. 3, pp. 2935–2942, 2026.

[12] L. Behnke, B. H. Do, S. Eristoff, and R. Kramer-Bottiglio, "Interaction behaviors of a vine robot in a pipe T-junction", in *IEEE Int. Conf. Soft Robot.*, 2024, pp. 498–503.

[13] M. T. Mason, "Mechanics and planning of manipulator pushing operations", *Int. J. Robot. Res.*, vol. 5, no. 3, pp. 53–71, 1986.

[14] K. M. Lynch and M. T. Mason, "Stable pushing: Mechanics, controllability, and planning", *Int. J. Robot. Res.*, vol. 15, no. 6, pp. 533–556, 1996.

[15] M. M. Coad, R. P. Thomasson, L. H. Blumenschein, N. S. Usevitch, E. W. Hawkes, and A. M. Okamura, "Retraction of soft growing robots without buckling", *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 2115–2122, 2020.

[16] M. Stilman and J. J. Kuffner, "Navigation among movable obstacles: Real-time reasoning in complex environments", *Int. J. Humanoid Robot.*, vol. 2, no. 4, pp. 479–504, 2005.

[17] J. D. Greer, T. K. Morimoto, A. M. Okamura, and E. W. Hawkes, "Series pneumatic artificial muscles (sPAMs) and application to a soft continuum robot", in *IEEE Int. Conf. Robot. Autom.*, 2017, pp. 5503–5510.

[18] S. Wang and L. H. Blumenschein, "Refined modeling for serial pneumatic artificial muscles enables model-based actuation design", in *IEEE Int. Conf. Soft Robot.*, 2024, pp. 800–807.

[19] S. M. LaValle and J. J. Kuffner, "Randomized kinodynamic planning", *Int. J. Robot. Res.*, vol. 20, no. 5, pp. 378–400, 2001.

[20] L. H. Blumenschein, A. M. Okamura, and E. W. Hawkes, "Modeling of bioinspired apical extension in a soft robot", in *Biomimetic Biohybrid Syst.*, 2017, pp. 522–531.

[21] R. L. Comer and S. Levy, "Deflections of an inflated circular-cylindrical cantilever beam", *AIAA J.*, vol. 1, no. 7, pp. 1652–1655, 1963.

[22] D. A. Haggerty, N. D. Naclerio, and E. W. Hawkes, "Characterizing environmental interactions for soft growing robots", in *IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 3335–3342.

[23] C. du Pasquier, S. Jeong, and A. M. Okamura, "Finite element modeling of pneumatic bending actuators for inflated-beam robots", *IEEE Robot. Autom. Lett.*, vol. 8, no. 11, pp. 7416–7423, 2023.

[24] J. Hwee, A. Lewis, A. Raines, and B. Hannaford, "Kinematic modeling of a soft everting robot from inflated beam theory", in *IEEE Int. Conf. Soft Robot.*, 2023, pp. 1–6.

[25] L. H. Blumenschein, M. Koehler, N. S. Usevitch, E. W. Hawkes, D. C. Rucker, and A. M. Okamura, "Geometric solutions for general actuator routing on inflated-beam soft growing robots", *IEEE Trans. Robot.*, vol. 38, no. 3, pp. 1820–1840, 2022.

[26] F. Fuentes, S. Diagne, Z. Kingston, and L. H. Blumenschein, "Linking exteroception and proprioception through improved contact modeling for soft growing robots", *Int. J. Robot. Res.*, 2026.

[27] P. Vartholomeos, Z. Wu, S. M. H. Sadati, and C. Bergeles, "Lumped parameter dynamic model of an eversion growing robot: Analysis, simulation and experimental validation", in *IEEE Int. Conf. Robot. Autom.*, 2024, pp. 12 734–12 740.

[28] B. Donald, P. Xavier, J. Canny, and J. Reif, "Kinodynamic motion planning", *J. ACM*, vol. 40, no. 5, pp. 1048–1066, 1993.

[29] Y. Li, Z. Littlefield, and K. E. Bekris, "Asymptotically optimal sampling-based kinodynamic planning", *Int. J. Robot. Res.*, vol. 35, no. 5, pp. 528–564, 2016.

[30] B. Huang, A. Boularias, and J. Yu, "Parallel Monte Carlo tree search with batched rigid-body simulations for speeding up long-horizon episodic robot planning", in *IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2022, pp. 1153–1160.

[31] N. Perrault, Q. H. Ho, and M. Lahijanian, "Kino-PAX: Highly parallel kinodynamic sampling-based planner", *IEEE Robot. Autom. Lett.*, vol. 10, no. 3, pp. 2430–2437, 2025.

[32] Y. Gao, A. Lu, and Z. Kingston, "Fast asymptotically optimal kinodynamic planning via vectorization", in *IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2026.

[33] M. R. Dogar and S. S. Srinivasa, "A planning framework for non-prehensile manipulation under clutter and uncertainty", *Auton. Robots*, vol. 33, no. 3, pp. 217–236, 2012.

[34] J. A. Haustein, J. King, S. S. Srinivasa, and T. Asfour, "Kinodynamic randomized rearrangement planning via dynamic transitions between statically stable states", in *IEEE Int. Conf. Robot. Autom.*, 2015, pp. 3075–3082.

[35] J. E. King, J. A. Haustein, S. S. Srinivasa, and T. Asfour, "Nonprehensile whole arm rearrangement planning on physics manifolds", in *IEEE Int. Conf. Robot. Autom.*, 2015, pp. 2508–2515.

[36] Muhayyuddin, M. Moll, L. E. Kavraki, and J. Rosell, "Randomized physics-based motion planning for grasping in cluttered and uncertain environments", *IEEE Robot. Autom. Lett.*, vol. 3, no. 2, pp. 712–719, 2018.

[37] K. Ren, L. E. Kavraki, and K. Hang, "Rearrangement-based manipulation via kinodynamic planning and dynamic planning horizons", in *IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2022, pp. 1145–1152.

[38] S.-G. Jeong, M. M. Coad, L. H. Blumenschein, M. Luo, U. Mehmood, J. H. Kim, A. M. Okamura, and J.-H. Ryu, "A tip mount for transporting sensors and tools using soft growing robots", in *IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2020, pp. 1002–1009.

[39] N. Badillo Perez, N. Pagliarani, M. Cianchetti, and R. D. Howe, *Tip manipulation in soft everting robots via wall retraction and deployable fingers*, arXiv:2609.23132, 2026.

[40] A. C. Torres, T. Marcucci, and E. W. Hawkes, *An efficient algorithm for minimum-pressure growth planning of vine robots*, arXiv:2609.18070, 2026.

[41] A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, and J. Z. Kolter, "Differentiable convex optimization layers", in *Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.

[42] R. Rubinstein, "The cross-entropy method for combinatorial and continuous optimization", *Methodol. Comput. Appl. Probab.*, vol. 1, no. 2, pp. 127–190, 1999.

[43] D. E. Stewart and J. C. Trinkle, "An implicit time-stepping scheme for rigid body dynamics with inelastic collisions and Coulomb friction", *Int. J. Numer. Methods Eng.*, vol. 39, no. 15, pp. 2673–2691, 1996.

[44] M. Anitescu and F. A. Potra, "Formulating dynamic multi-rigid-body contact problems with friction as solvable linear complementarity problems", *Nonlinear Dyn.*, vol. 14, no. 3, pp. 231–247, 1997.

[45] M. Anitescu, "Optimization-based simulation of nonsmooth rigid multibody dynamics", *Math. Program.*, vol. 105, pp. 113–143, 2006.